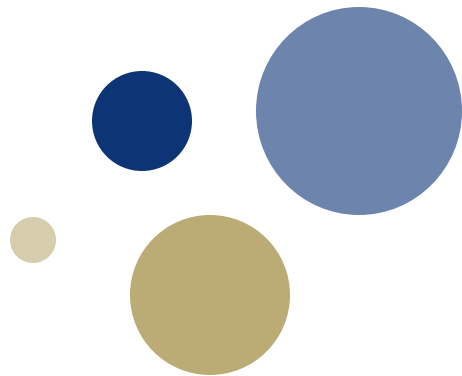




NTNU

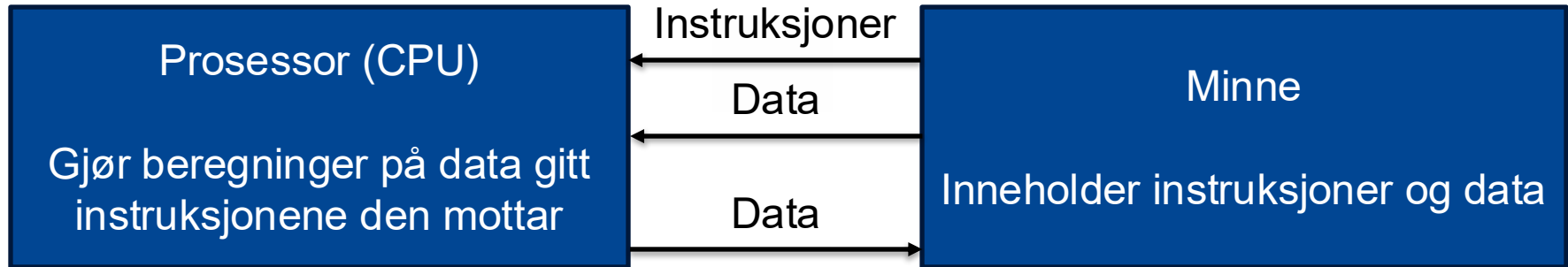
Kunnskap for en bedre verden



Forelesning 12: Parallele datamaskiner og akseleratorer

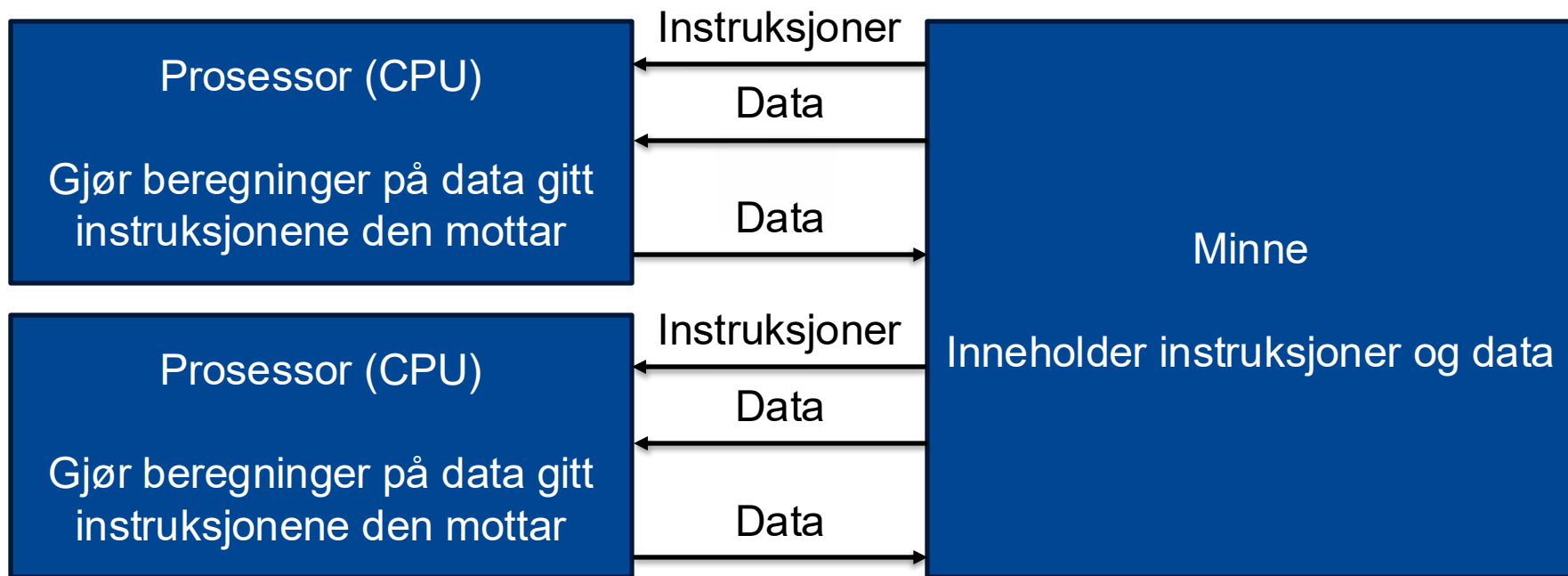
TDT4160 Datamaskiner

Prinsippet om lagrede program



Prinsippet om lagrede program

I dag: Parallele datamaskiner og akseleratorer





Tema 7.1 (Kapittel 6.1, 6.2, 6.3)

YTELSE OG FLYNNNS TAKSONOMI

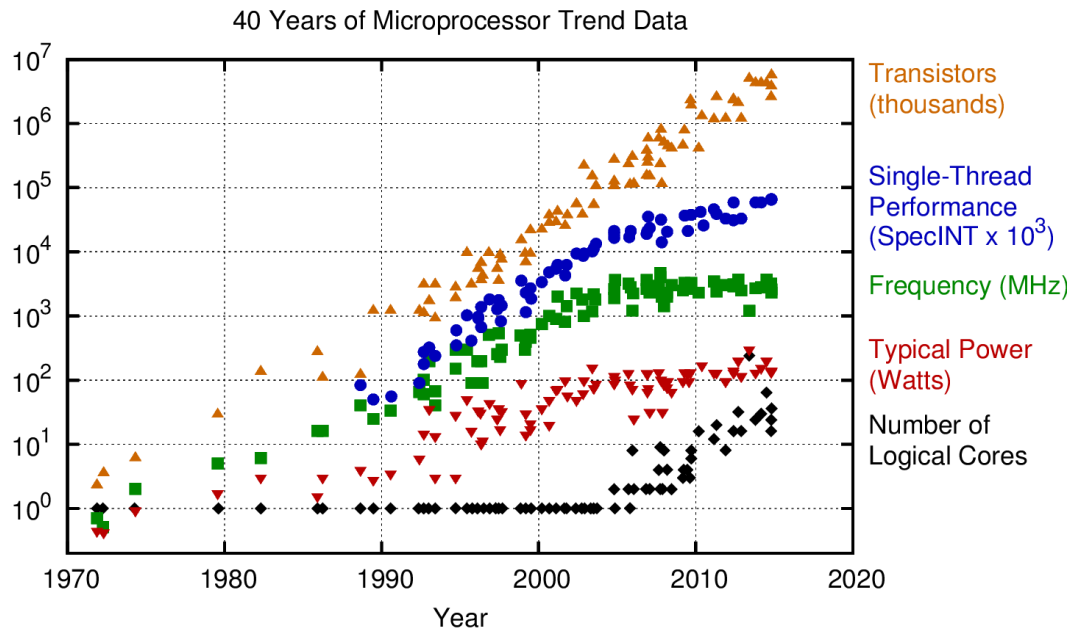
Læringsutbytte T7.1

- Studenten skal kunne forklare hvorfor det er enklere å utnytte **parallelitet** mellom **uavhengige program** enn **innad i ett program**.
- Studenten skal kunne bruke **Amdahls lov** til å analysere **skalerbarheten** til parallelle programmer.
- Studenten skal kjenne til konseptene **sterk og svak skalering** og kunne bruke dem til å vurdere skalerbarheten til ulike applikasjoner.
- Studenten skal kunne forklare **Flynns taksonomi** og kunne gi eksempler på SISD, SIMD, og MIMD maskiner.
- Studenten skal kunne bruke Roofline-modellen og konseptet operasjonsintensitet til å analysere hvorvidt en applikasjon er minnebundet eller beregningsbundet på en gitt datamaskin.

Hvorfor er stort sett alle datamaskiner parallelle?

$$Power \propto 1/2 \times Capacitive\ load \times Voltage^2 \times Frequency\ switched$$

- Dagens prosessorer er begrenset av effektforbruk
 - ... og effektforbruket øker lineært med klokkefrekvensen
- Vi kan ikke (lenger) øke klokkefrekvensen
 - ... og dette begrenser ytelsesøkningen til en enkelt prosessor
- Antallet transistorer på per brikke har imidlertid fortsatt å doble seg hvert andre år (Moore's lov)
 - ... og effektforbruket vokser lineært med kapasitansen til de transistorene som bytter tilstand (som ikke er på langt nær alle)
- Løsning: Vi legger til flere prosessorkjerner
 - Høyere ytelse uten (vesentlig) økt effektforbruk
 - Udfordring: Bedre ytelse krever parallelitet på programvarenivå



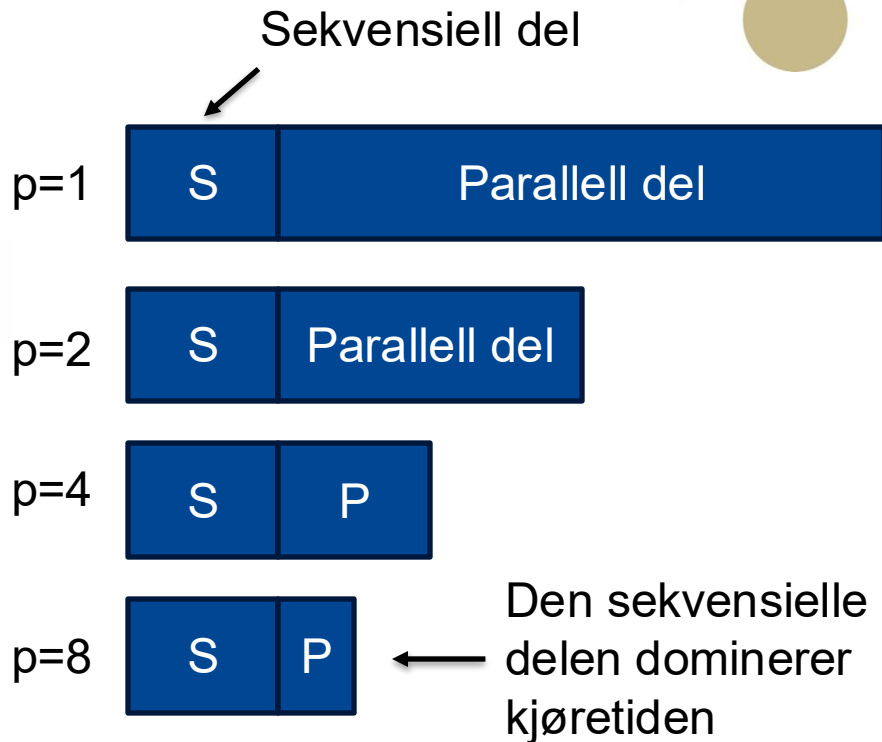
Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2015 by K. Rupp

Parallellitet på programvarenivå

- Vi skiller mellom parallellitet mellom (uavhengige) program og mellom (avhengige) oppgaver i ett program
 - Uavhengige: Programmer som kjøres i parallell (f. eks. «Process-Level Parallelism»)
 - Avhengige: Et parallelt program
- Det er trivielt å utnytte parallellitet mellom uavhengige program
 - Typisk eksempel: Du skal gjøre den samme jobben med n forskjellige innverdier
- Det er vanskelig(ere) å utnytte parallellitet til å løse én oppgave
 - ... fordi vi da må passe på at alle avhengigheter respekteres når vi skriver koden

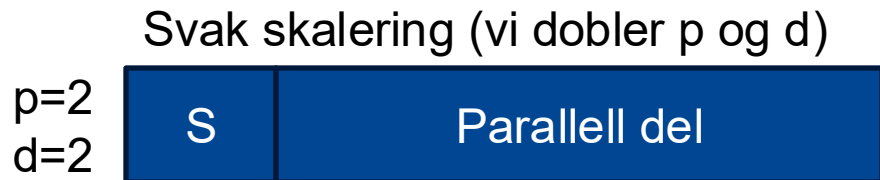
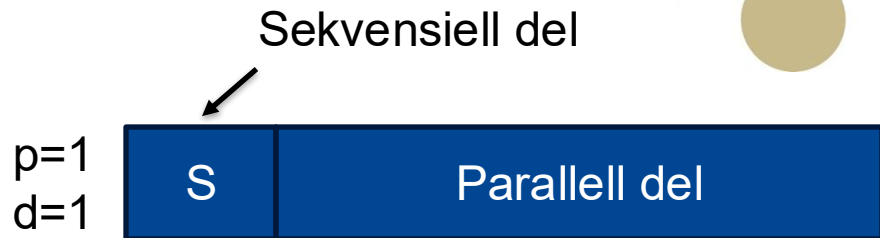
Amdahl's lov for parallelle program

- Amdahl: Et parallelt program består av en sekvensiell og en parallell del
- Når vi øker antall prosessorer
 - ... fordeles kjøretiden til den parallelle delen jevnt utover alle prosessorene
 - ... mens kjøretiden til den sekvensielle delen forblir uendret



Sterk og svak skalering

- Amdahls lov antar at programmet gjør samme mengde arbeid selv om man øker antallet prosessorer
 - Med andre ord: vi beholder samme problemstørrelse
 - Dette kalles sterk skalering («strong scaling»)
- Vi kan også anta at problemstørrelsen vokser i takt med antallet prosessorer
 - Da holder forholdet mellom den sekvensielle og den parallelle delen seg omtrent konstant
 - Dette kalles svak skalering («weak scaling»)
- Hvilket skaleringsmønster som best beskriver et program er ofte gitt av situasjonen det brukes i

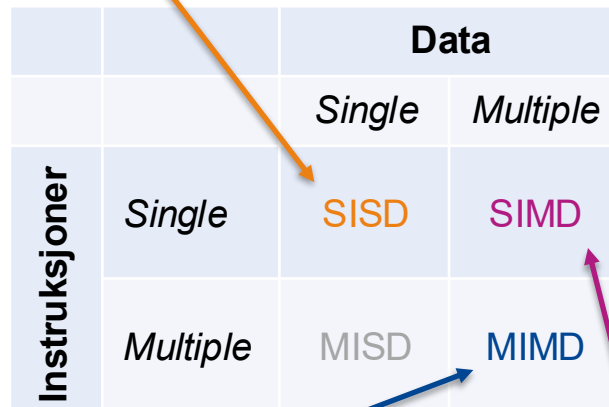


Flynns taksonomi

- Hvilke typer parallelle datamaskiner kan man lage?

Eksempel: En datamaskin med enkelt prosessor

- To dimensjoner:
 - Har vi en (S) eller flere (M) instruksjonsstrømmer (I)?
 - Har vi en (S) eller flere (M) datastrømmer (D)?



The diagram shows a 2x2 matrix for Flynn's taxonomy. The columns are labeled 'Data' (Single, Multiple) and the rows are labeled 'Instruksjoner' (Single, Multiple). The cells contain: SISD (Single Instruction, Single Data), SIMD (Single Instruction, Multiple Data), MISD (Multiple Instruction, Single Data), and MIMD (Multiple Instruction, Multiple Data). An orange arrow points from the text 'Eksempel: En datamaskin med enkelt prosessor' to the SISD cell. A blue arrow points from the text 'Eksempel: En datamaskin med mange prosessorer' to the MIMD cell. A purple arrow points from the text 'Eksempel: SIMD-instruksjoner' to the SIMD cell.

		Data	
		Single	Multiple
Instruksjoner	Single	SISD	SIMD
	Multiple	MISD	MIMD

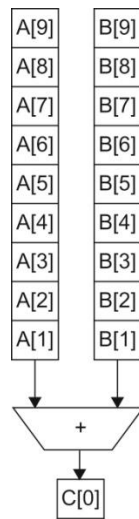
Eksempel: En datamaskin med mange prosessorer

Eksempel:
SIMD-instruksjoner

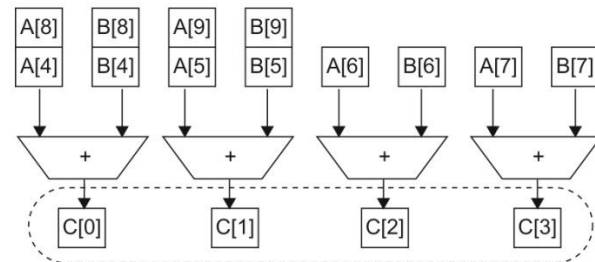
Hvordan utnytte SIMD?

- Fordel: Vi utfører samme instruksjon på mange dataelementer
 - ... så arbeidet gjort per instruksjon øker betraktelig
 - Vi spesifiserer også at elementene er uavhengige av hverandre
- Stort sett alle høytytelsesprosessorer har i dag SIMD-instruksjoner
 - ... som som gjør samme operasjon på flere dataelementer
- Vi har også vektorprosessorer
 - Disse er typisk mer fleksible i hvordan de støtter vektoroperasjoner (f. eks. variabel vektorlengde)
- Forgreningsinstruksjoner er en utfordring for alle SIMD-arkitekturer

$$C = A + B$$



(a)



(b)

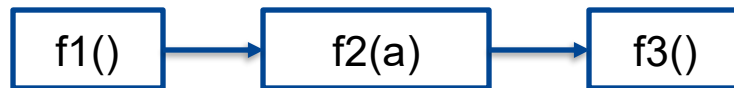


Det store bildet

HVORFOR ER MODERNE DATAMASKINER HETEROGENE?

Hva er nytt?

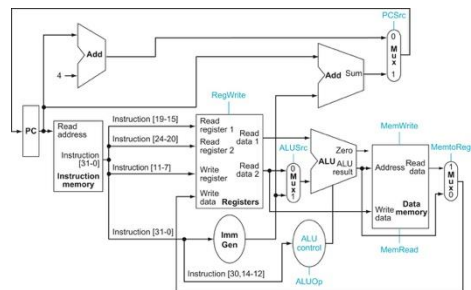
- Vi har så langt kun sett på serielle program
- Vi har nå parallell programvare
... og vi må bygge maskiner som kan utnytte dette



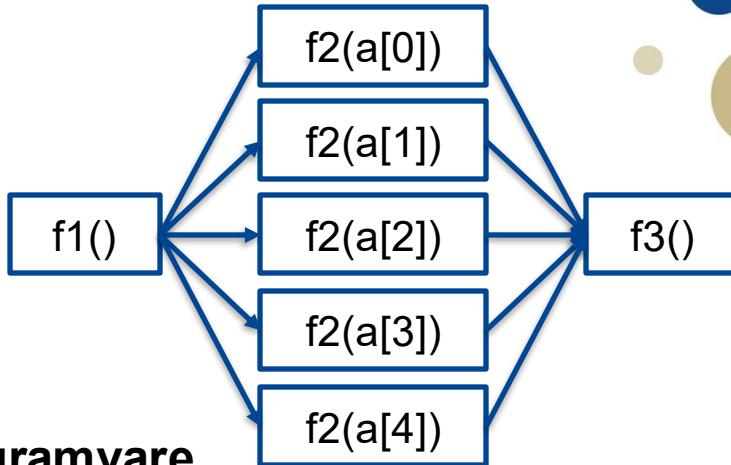
Programvare

Maskinvare

ISA



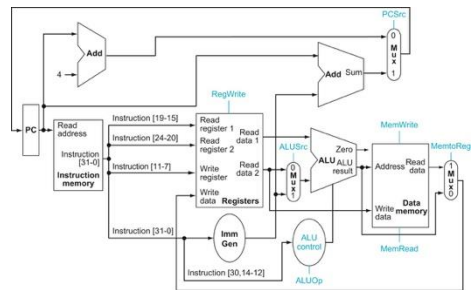
- ... og vi må bygge
maskiner som kan
utnytte dette



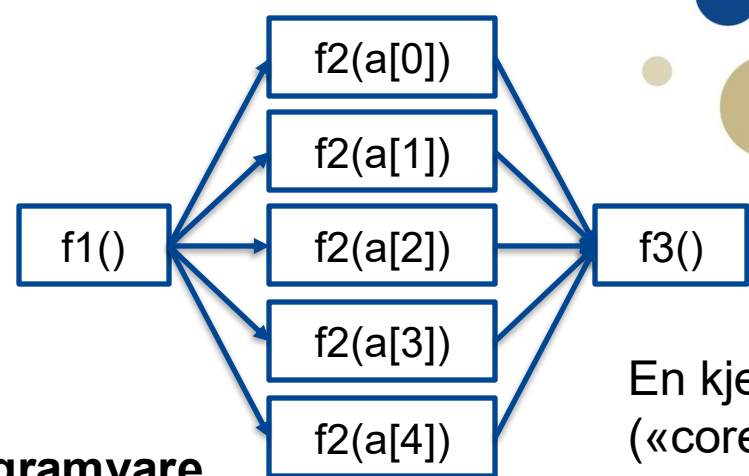
Programvare

Maskinvare

ISA



- ... og vi må bygge maskiner som kan utnytte dette

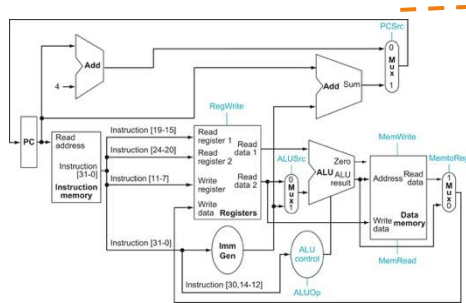


En kjerne («core»)

Programvare

Maskinvare

ISA



CPU

CPU

GPU

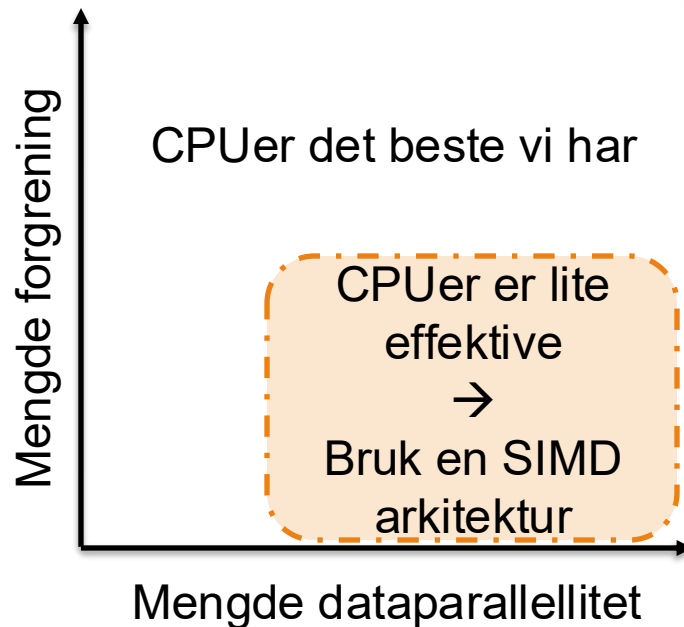
CPU

CPU

DSA

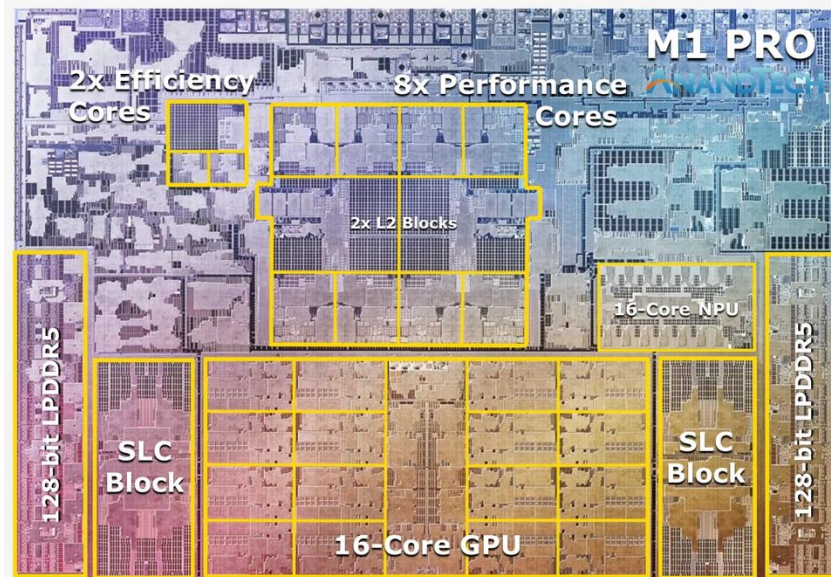
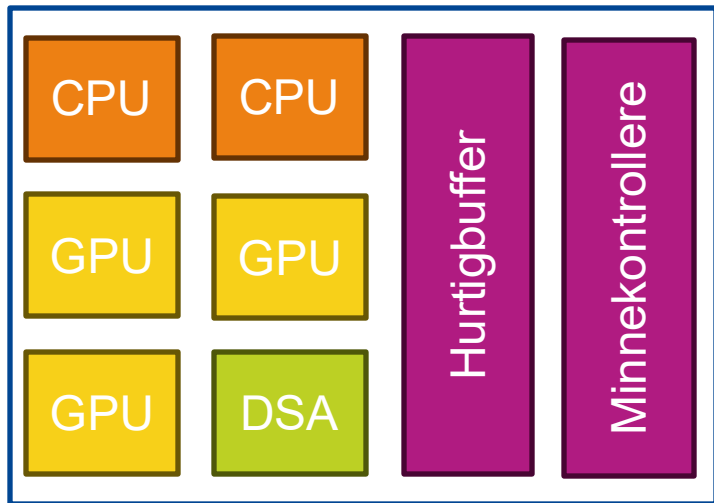
Hva er mulighetsrommet?

- Vi kan lage prosessorkjerner som fokuserer på enten SISD eller SIMD
 - ... og så kobler vi typisk sammen en drøss av dem for å utnytte MIMD
 - (Moderne datamaskiner bruker alle triksene i boka)
- Prosessorene (CPUene) vi har sett på så langt er de mest effektive SISD-arkitekturerne vi kjenner til
 - ... men de er relativt lite effektive for dataparallell kode med lite (eller regulær) forgreining
- Nøkkelobservasjon: **SIMD-arkitekturer er gode på det CPUer er dårlige til**
 - ... og datamaskinen blir dermed mest effektiv hvis vi kombinerer dem
 - Da får vi en heterogen datamaskin



Hvordan ser dette egentlig ut?

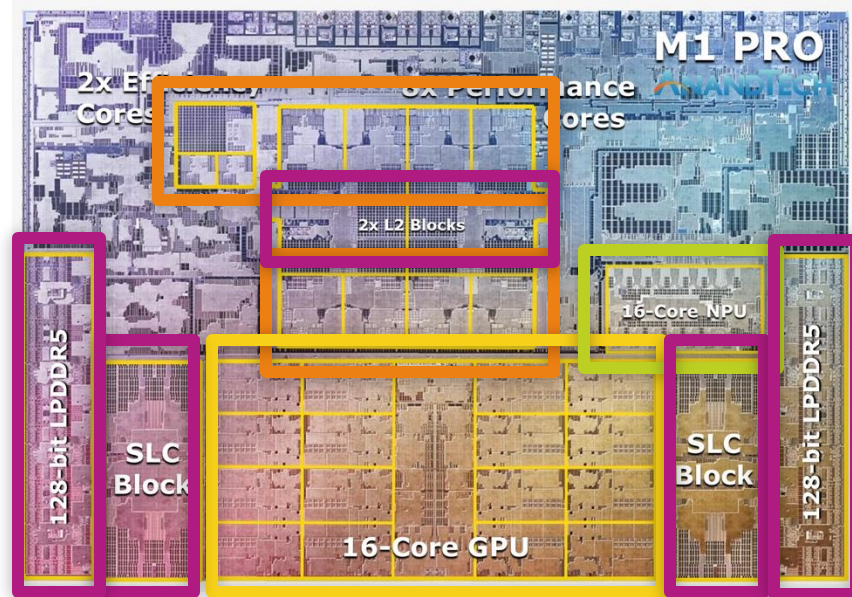
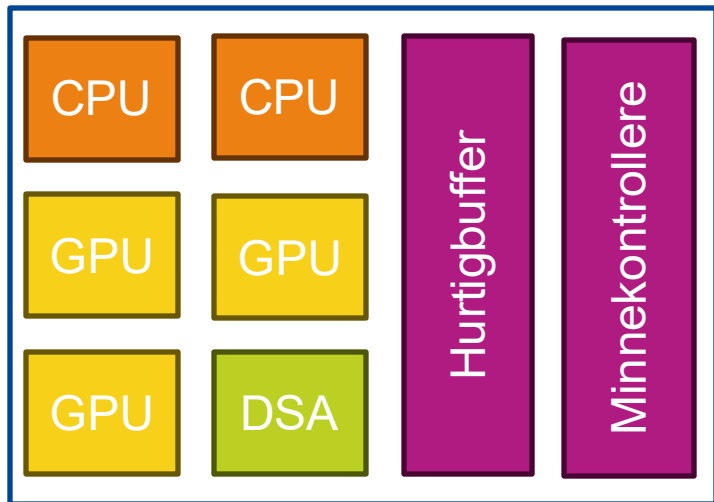
Prosessorbrikken



Prosessorbrikken i min laptop: Apple M1 Pro

Hvordan ser dette egentlig ut?

Proessorbrikken



Proessorbrikken i min laptop: Apple M1 Pro



Tema 7.2 (Kapittel 2.11, 5.10, 6.4, 6.5, 6.8)

MULTIPROSESSORER

Læringsutbytte T7.2

- Studenten kan beskrive den overordnede arkitekturen til **multiprosessorer** med **delt minne** og **distribuert minne** samt kjenne til deres styrker og svakheter.
- Studenten skal kunne beskrive den overordnede arkitekturen til en **flerkjerneprosessor** og kunne motivere for hvorfor arkitekturen er som den er.
- Studenten skal kunne forklare hvorfor **synkronisering** er nødvendig og de grove trekkene i hvordan synkronisering implementeres.
- Studenten skal kjenne til behovet for å holde **hurtigbufferne koherente** og de grove trekkene i hvordan det løses.
- Studenten skal kunne beskrive **minnekonsistensproblemet** og kjenne til de grove trekkene i hvordan det løses.

Hvorfor multiprosessorer?



- Parallellitet mellom uavhengige program er vanlig
- Parallele program med irregulær kontrollflyt er krevende for SIMD-arkitekturer
- Akselererbare applikasjoner trenger også CPUer (Amdahl's lov)
 - ... og fra dette perspektivet er multiprosessoren infrastruktur akseleratorene trenger for å yte best mulig
- Kanskje viktigst: Parallellprogrammering er vanskelig
 - ... men et program for parallelle CPUer kan likne ganske mye på det serielle programmet man allerede har skrevet (se OpenMP eksemplet i boka)

Eksempel: OpenMP

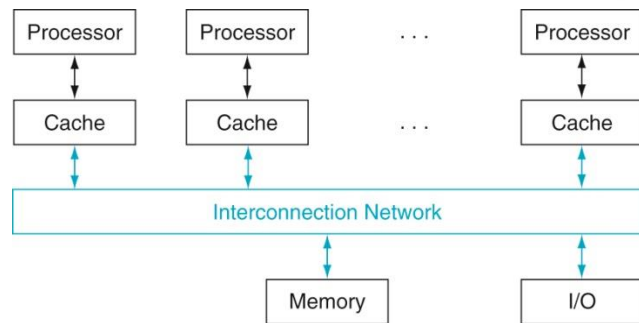
La oss kjøre litt kode



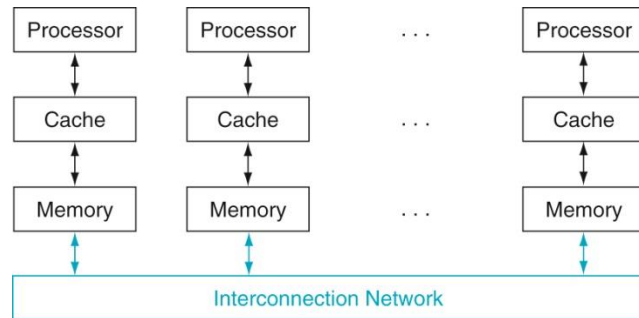
Typer multiprosessorer

- Hvor mange prosessorbrikker er det i systemet?
 - Hvis en: Flerkjerneprosessor («multi-core processor»)
 - Hvis mer enn en: Multiprosessor («multiprocessor»)
 - Alle multiprosessorer i dag er samlinger av flerkjerneprosessorer
- Deler prosessorene det fysiske minnerommet?
 - Ja: Delt minne («Shared Memory Multiprocessor (SMP)»)
 - Nei: Distribuert minne («Distributed-memory multiprocessor»)

Delt minne



Distribuert minne



Flerkjerneprosessorer



- Flerkjerneprosessorer er delt minne arkitekturer
 - ... og deler også typisk siste nivå hurtigbuffer («Last Level Cache (LLC)») mellom kjernene
- Hver prosessorkjerne har typisk to nivåer med private hurtigbuffer
 - Nivå 1 («L1») er nærmest prosessorkjernen og liten (~32KiB) for å få trefftiden ned i ~3 klokkesyklus
 - L2 er større (~1MB)
- De private L2 hurtigbufferene er koblet til LLC via et «Network-on-Chip (NoC)»
- Bom i LLC går til minnekontrollerne som tar seg av lesing og skriving til hovedminnet (DRAM)

Eksempel: Synkronisering

Adresse	Verdi
8	7
4	3

Prosess 0

0 → Minnet[4]
Minnet[8] → x1

Prosess 1

1 → Minnet[8]
Minnet[4] → x2

Hvilke verdier har vi i x1 og x2 når koden over har kjørt?



Delt minne: Synkroniseringsinstruksjoner

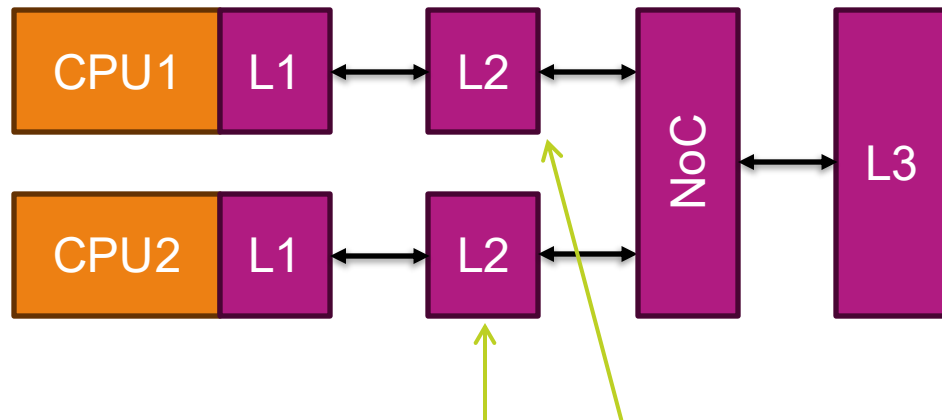


- Enkelte ting må utføres i rekkefølge (synkroniseres)
 - Vi kaller den delen av koden som må utføres i rekkefølge for en kritisk seksjon («critical section»)
 - Eksempel: Når vi legger sammen delsummene fra hver tråd i eksempelprogrammet
- Synkroniseringsmekanismer blir mer effektive hvis de støttes av spesialinstruksjoner
 - Fundamentalt må vi først lese og så skrive en minnelokasjon samt kunne garantere at ingenting har skjedd mellom leseoperasjonen og skriveoperasjonen
 - Typisk implementert med to instruksjoner: «Load reserved» and «store conditional»

Delt minne: Konsistens og koherens

TDT4260

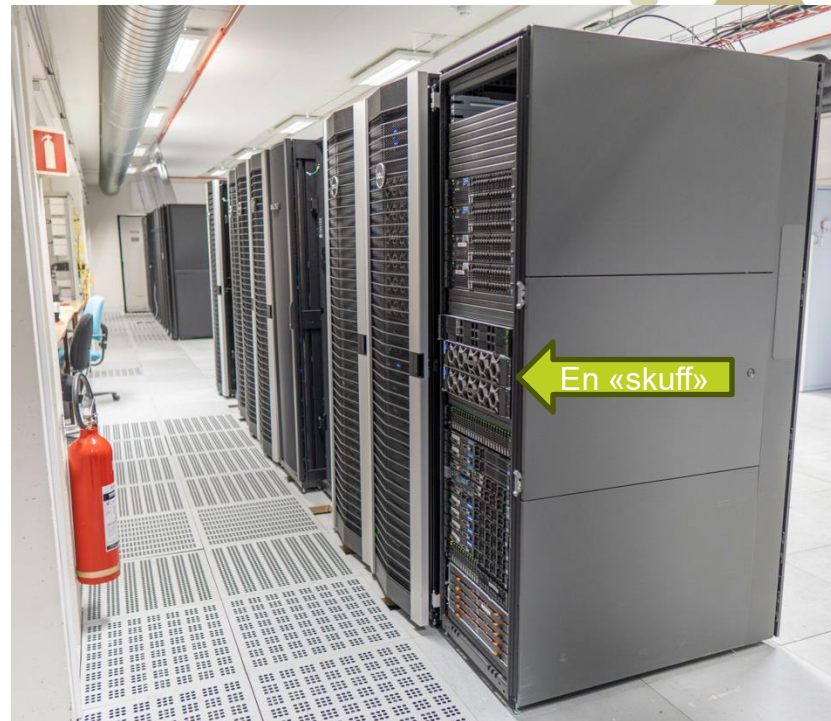
- Koherens («coherence»):
 - Alle prosessorer må se operasjoner på samme dataelement i samme rekkefølge
 - Vi må ha maskinvarestøtte som garanterer denne rekkefølgen («cache coherence protocols»)
 - Vi bryr oss ikke om hvilken rekkefølge dette er så lenge alle er enige
- Konsistens («consistency»)
 - Regler for de lovlige rekkefølgene på operasjoner til ulike dataelement
 - Av alle mulige rekkefølger på operasjoner, er noen lov og noen ikke lov → og maskinen må passe på at bare lovlige rekkefølger oppstår



Hvis en av disse skriver en verdi som den andre allerede har lest, har vi et koherensproblem

Distribuert minne: Clustere

- Hver «skuff» inneholder (flere) delt minne multiprosessorer
 - ... som er koblet sammen med et (kraftig) nettverk
 - Vi kaller hver delt minne multiprosessor en node
- Prosessene på ulike noder kommuniserer ved å sende meldinger over nettverket («message passing»)
- Jo mer applikasjonen kommuniserer mellom nodene, jo kraftigere nettverk trenger vi
 - ... og da begynner vi å nærme oss superdatamaskinene (se <https://top500.org/>)



NTNUs Idun cluster
(~8580 CPU kjerner, ~234 GPUer)

Distribuert minne: «Warehouse-Scale Computers»

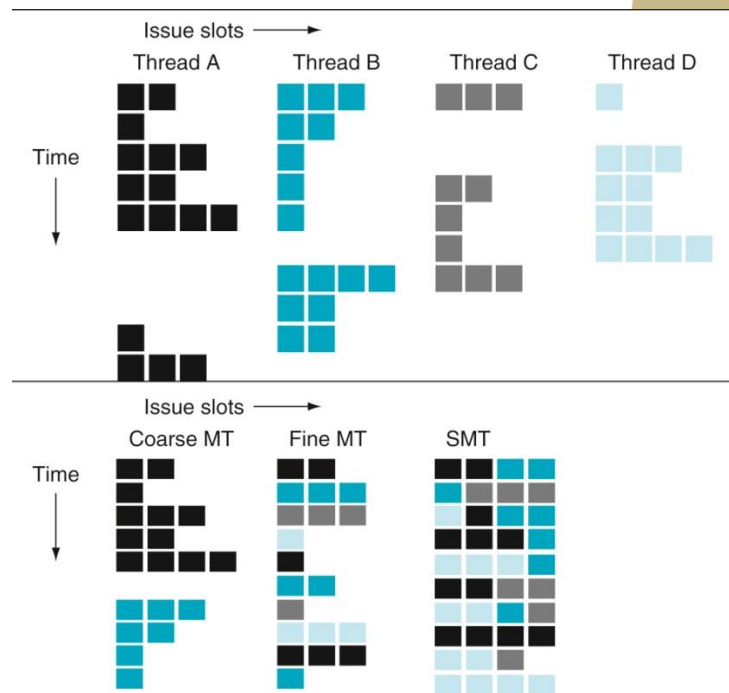
- Multiprosessorer med distribuert minne kan skaleres så mye man bare vil
 - Googles datasenter i Skien er dimensjonert for et effektforbruk på 860 MW
 - 860 MW i ett år $\rightarrow$ 7.5 TWh
 - Elektrisk energiforbruk for husholdninger i Trøndelag i 2023 $\rightarrow$ 3.5 TWh
- Datasenterapplikasjoner er typisk drevet av forespørsler fra brukere (i global skala)
 - Ditt søk er uavhengig av alle andre søk
 - Datastrukturer som bare leses er enkle å distribuere (i søk: indeksen)



Googles datasenter i Eemshaven, Nederland

«Hardware Multithreading»

- Observasjon: Noen minneintensive applikasjoner utnytter ressursene i prosessorkjernen dårlig
- Løsning: Vi utvider prosessoren slik at den kan kjøre mer enn en prosess av gangen og får dermed flere parallelle minneforespørsler
 - Hver maskinvaretråd må ha sin egen PC og registerfil
 - De andre enhetene i prosessorkjernen kan enten dupliseres eller deles
- Når bytter vi prosess?
 - «Coarse MT»: Når en prosess stanser lenge
 - «Fine MT»: Hver klokkesykel (blant de prosessene som ikke har stanset)
 - «Simultaneous multithreading (SMT)»: Prosessene kjører samtidig
- Man pleier ofte å skru av «hardware multithreading»
 - ... fordi beregningsintensive prosesser vil bli mye tregere





Tema 7.3 (Kapittel 6.6 og 6.7)

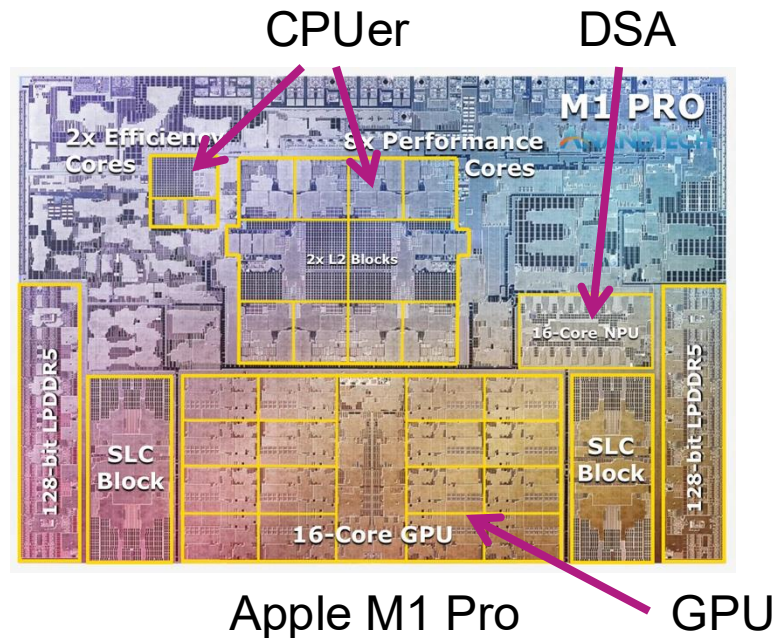
AKSELERATORER

Læringsutbytte T7.3

- Studenten skal vite hva en **heterogen datamaskin** er og kunne forklare hvorfor datamaskiner med høy ytelse typisk er heterogene.
- Studenten skal kunne forklare den **overordnede arkitekturen** til **grafikkprossessorer (GPUer)** samt kunne beskrive **programmeringsmodellen** som benyttes når man bruker GPUer til generell beregning.
- Studenten skal kunne beskrive **likheter og forskjeller** mellom **GPUer**, **vektorprossessorer**, og **SIMD-instruksjoner**.
- Studenten skal kunne forklare hva en **domene-spesifikk akselerator (DSE)** er og motivere for hvorfor disse kan oppnå høyere ytelse enn mer generelle arkitekturer (altså CPUer og GPUer).

Dagens datamaskiner er heterogene

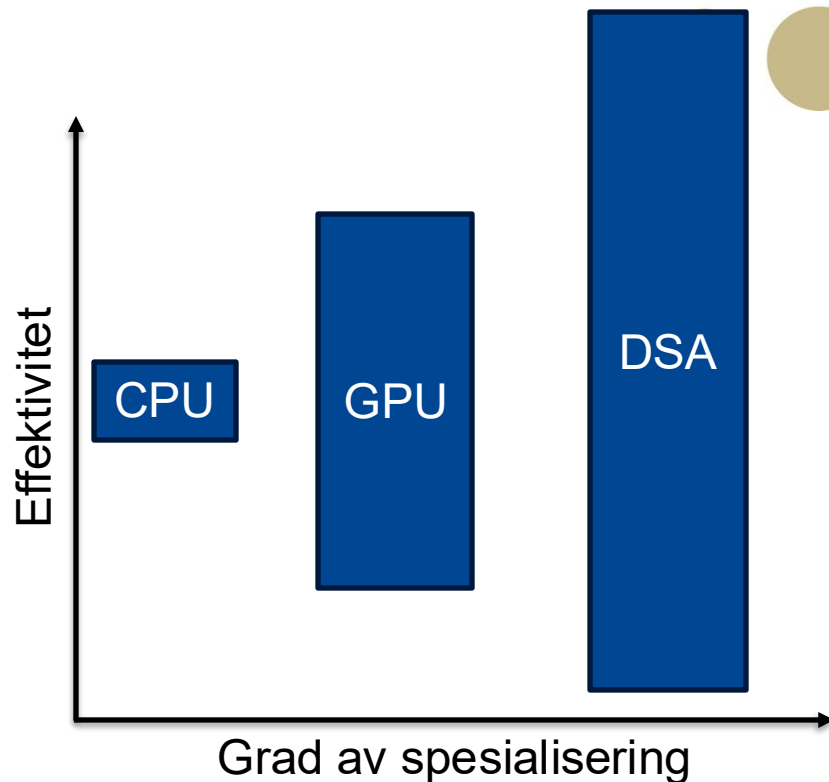
- Heterogen = har beregningsenheter med ulike egenskaper
 - Dette er i motsetning til en homogen maskin der alle beregningsenhetene (prosessorene) er like
- Å hente og dekode en instruksjon kan forbruke 10X til 4000X så mye energi som å utføre en addisjon [1]
 - ... og CPUer gjør (relativt) lite arbeid per instruksjon
- Vi laget CPUene for å være generelle og dette er kostnaden
 - ... og fellesnevneren for akseleratorer er at de er bygget for å utnytte SIMD-parallellitet



[1] W. J. Dally, Y. Turakhia, and S. Han, "Domain-specific hardware accelerators," *Communications of the ACM*, vol. 63, no. 7, pp. 48–57, 2020.

Generalitet versus spesifisitet

- En beregningsenhet er generell hvis den er (relativt) effektiv for mange ulike applikasjoner
 - Eksempel: CPUer
- En beregningsenhet er spesialisert hvis den er spesielt effektiv for en (eller flere) klasser av applikasjoner
 - Eksempel: Grafikkprosessorer (GPUer) og domenespesifikke akseleratorer (DSA)
- Rett enhet på rett applikasjon
 - → betydelig effektivitetsforbedring (~15000X over en CPU [1])



Domenespesifikke akseleratorer (DSA)

- Vi lager maskinvare som passer spesielt godt til beregnings og kommunikasjonsmønsteret i en «viktig» (klasse) applikasjon(er)
- Fordel: Høy effektivitet
 - Eksempel Google TPuv4: 1.2X til 1.7X raskere enn en tilsvarende GPU med et 1.3X til 1.9X lavere effektforbruk [2]
- Ulempe: Vi må bygge egne beregningsenheter for «alle» problemer
 - ... men dette gir mening når man lager en KI-akselerator til Googles datasentere

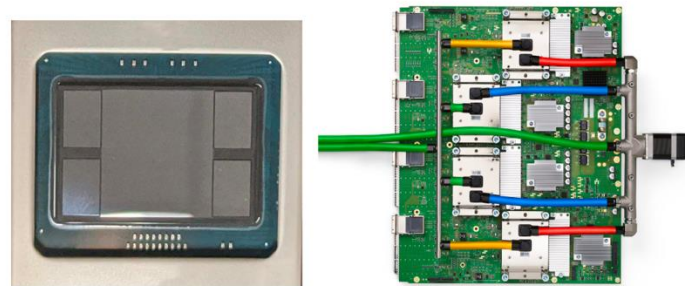


Figure 2: The TPU v4 package (ASIC in center plus 4 HBM stacks) and printed circuit board with 4 liquid-cooled packages. The board's front panel has 4 top-side PCIe connectors and 16 bottom-side OSFP connectors for inter-tray ICI links.

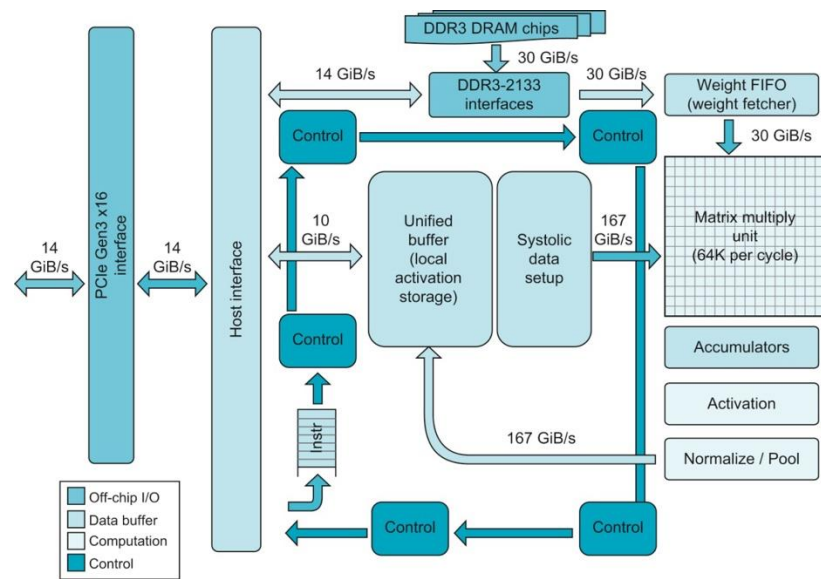


Figure 3: Eight of 64 racks for one 4096-chip supercomputer.

Bildet er hentet fra Jouppi et al. [2]

DSA: Designprinsipper

- Tilpass minnesystemet til domenet
 - TPU: Dedikerte minner til vektene
- Bruk ressursene vi sparer på forenklet instruksjonsbehandling til ALUer og minner
 - TPU: 64Ki ALUer i «Matrix Multiply Unit»
- Tilpass arkitekturen til parallelliteten som er tilgjengelig i domenet
 - KI: Mye matrisemultiplikasjon
- Ikke bruk flere bit i datatypen enn du må
 - KI: 4, 8, eller 16 bit er vanlig
- Bruk domenespesifikke språk (og programmeringsmodeller)
 - KI: For eksempel TensorFlow



TPUv1 fra boka

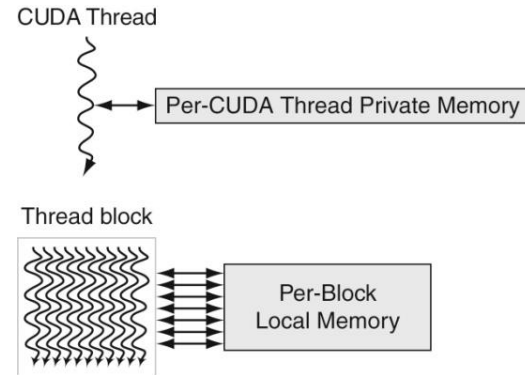
Hvorfor har vi ikke DSAer for alle applikasjoner?



- Vi må lage en komplett datamaskin med systemprogramvare for hvert eneste domene
 - ... og dette er ekstremt ressurskrevende
- Det er behov for en maskin som utnytter dataparallellitet effektivt
 - ... men samtidig er generell nok til at den er effektiv i mange ulike domener
- Løsning: Grafikkprosessorer (GPUer)
 - ... som først ble laget for grafikk men nå brukes til alt mulig
 - Arm snakket om GPUer for grafikk i går (mandag)

GPU: Programmeringsmodell

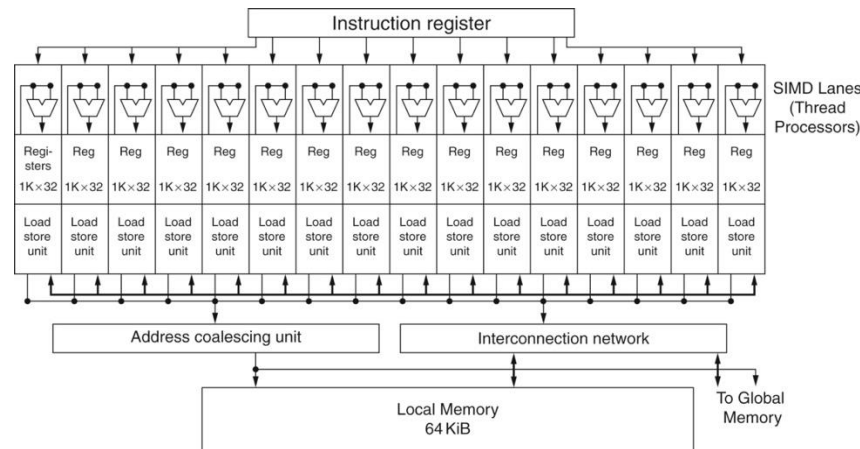
- GPUer fokuserer på gjennomstrømming (og ikke kjøretid)
 - (Hvis du bryr deg om kjøretid, kan du bruke en CPU)
- Den fundamentale enheten er en tråd
 - ... og trådene samles i grupper som gjør samme operasjon på ulike data (SIMD)
 - Gruppene kalles «warps» (av Nvidia)
- «Warps» samles i «Thread Blocks» som er garantert å kjøre på samme GPU-kjerne
 - ... og en GPU-kjerne kalles en «Streaming Multiprocessor (SM)»
- Hver SM har tilgang til flere «Thread Blocks» til en hver tid
 - ... og hvis en av dem stanser bytter den bare til en annen «Thread Block»
 - GPUer bruker altså parallellitet på programvarenivå til å skjule latens → vi må ha vesentlig mer parallellitet i programmet enn det er i maskinen



GPU mikroarkitektur

TDT4260

- Vi velger en «warp» som er klar til å utføres
- Alle trådene i en «warp» utfører samme operasjon på ulike data
 - ... som for alle praktiske formål gjør en «warp» til en SIMD-instruksjon



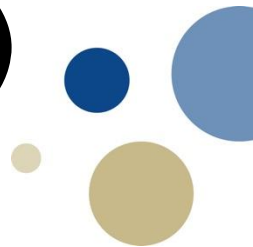
Forgreningsinstruksjoner i SIMD arkitekturer



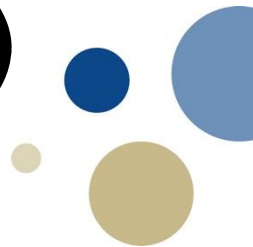
- Effektivitetsforbedringen til SIMD-instruksjoner, vektorprosessorer, GPUer, og DSAer kommer fra at de lar én operasjon gjelde (mange) dataelementer
- Dette fungerer godt så lenge forgreining skjer mellom SIMD-instruksjoner
 - ... men forgreining inne i en SIMD-instruksjon er problematisk

```
for(int i=0;i<N;i++) {  
    if(i % 2 == 0) {  
        data[i] += 1;  
    }  
    else{  
        data[i] = 0;  
    }  
}
```


Eksempel: SIMD og forgreining (1)



Eksempel: SIMD og forgreining (2)





Tema 7.1 (Kapittel 6.11)

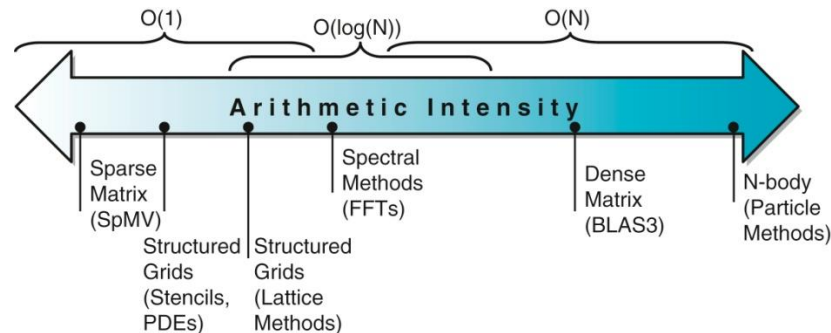
ROOFLINE-MODELLEN

Læringsutbytte T7.1

- Studenten skal kunne forklare hvorfor det er enklere å utnytte parallellitet mellom uavhengige program enn innad i ett program.
- Studenten skal kunne bruke Amdahls lov til å analysere skalerbarheten til parallelle programmer.
- Studenten skal kjenne til konseptene sterk og svak skalering og kunne bruke dem til å vurdere skalerbarheten til ulike applikasjoner.
- Studenten skal kunne forklare Flynns taksonomi og kunne gi eksempler på SISD, SIMD, og MIMD maskiner.
- Studenten skal kunne bruke **Roofline-modellen** og konseptet operasjonsintensitet til å analysere hvorvidt en applikasjon er minnebundet eller beregningsbundet på en gitt datamaskin.

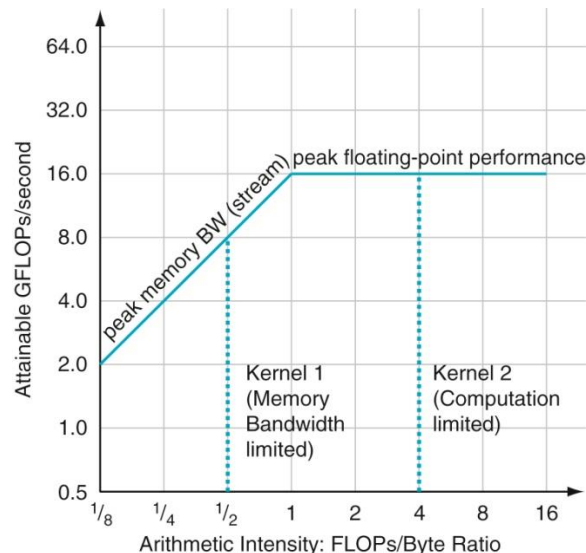
Operasjonsintensitet

- Vi kan karakterisere applikasjoner basert på deres operasjonsintensitet
 - = antallet operasjoner utført per byte hentet fra minnet
 - «Operational intensity» eller «Arithmetic intensity»
- Lavere operasjonsintensitet
→ færre operasjoner per byte → applikasjonen er mer minnebundet



Roofline

- Vi kan bruke operasjonsintensitet til å forstå om ytelsen til en applikasjon er begrenset av beregning eller minnebåndbredde på en gitt maskin
 - ... og det er dette Roofline-modellen gjør
- Hvis operasjonsintensiteten er høy nok er applikasjonen beregningsbegrenset
 - Hvis ikke, er minnebåndbredden begrensningen
- Vi kan også bruke Roofline til å analysere hvor godt en implementasjon utnytter maskinen
 - Hvor langt unna maskinens maksimale ytelse er vi?





Tema 7: Parallele datamaskiner

OPPSUMMERING

Parallele datamaskiner

- Å bare bruke én prosessor er en fundamental begrensning
- Vi kan skalere opp datamaskinen ved å legge til flere prosessorer (CPUer)
 - → Flerkjerneprosessorer og multiprosessorer
- Vi kan lage beregningsenheter som utnytter parallellitet mer effektivt enn CPUer
 - → Grafikkprosessorer (GPUer) og domene-spesifikke akseleratorer (DSAer)
- Vi kan kombinere ulike beregningsenheter i samme system
 - → Heterogene datamaskiner
 - I dag: De aller fleste høyytelsesdatamaskiner heterogene